



Analisis Klaster

Density-Based Methods, Bi-Clustering, dan Evaluasi Model

Pertemuan 10



Chapter 10. Cluster Analysis: Basic Concepts and Methods

- Cluster Analysis: Basic Concepts
- Partitioning Methods
- Hierarchical Methods
- **Density-Based Methods**
- **Bi-Clustering**
- **Evaluation of Clustering**
- Summary



Density-Based Methods



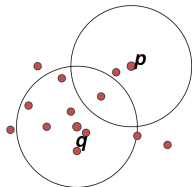
Density-Based Clustering Methods

- Pengelompokan berdasarkan kepadatan (kriteria kluster lokal), seperti titik yang terhubung dengan kepadatan
- Density/kepadatan = Jumlah titik dalam radius tertentu r (Eps)
- Fitur utama:
 - Temukan kluster dengan bentuk bermacam-macam
 - Menangani *noise*
 - Satu *scan*
 - Perlu parameter kepadatan sebagai kondisi terminasi
- Beberapa studi menarik:
 - DBSCAN: Ester, et al. (KDD'96)
 - OPTICS: Ankerst, et al. (SIGMOD'99)
 - DENCLUE: Hinneburg & D. Keim (KDD'98)
 - CLIQUE: Agrawal, et al. (SIGMOD'98) (*more grid-based*)



Density-Based Clustering: Basic Concepts

- Dua parameter:
 - **Eps**: Radius maksimum *neighbourhood*
 - **MinPts**: Jumlah titik minimum dalam *Eps-neighbourhood* dari titik itu
- $N_{Eps}(q)$: $\{q \text{ termasuk dalam } D \mid dist(p, q) \leq Eps\}$
- **Directly density-reachable**: Titik p dapat dijangkau secara langsung dari titik q dengan mengacu Eps , $MinPts$ jika
 - p termasuk dalam $N_{Eps}(q)$
 - Kondisi titik *core*:
 $|N_{Eps}(q)| \geq MinPts$



MinPts = 5

Eps = 1 cm

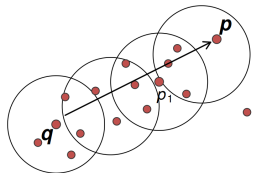
Titik adalah titik *core* jika memiliki lebih dari jumlah titik yang ditentukan ($MinPts$) dalam Eps .



Density-Reachable and Density-Connected

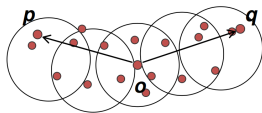
- Density-reachable:

Titik p adalah **density-reachable** dari titik q dengan mengacu Eps , $MinPts$ jika ada rantai titik $p_1, \dots, p_n$, $p_1 = q$, $p_n = p$ sedemikian rupa sehingga p_{i+1} dapat dijangkau kepadatannya langsung dari p_i



- Density-connected:

Satu titik p adalah **density-connected** ke suatu titik q dengan mengacu Eps , $MinPts$ jika terdapat suatu titik o sedemikian rupa sehingga keduanya, p dan q , *density-reachable* dari o dengan mengacu Eps dan $MinPts$

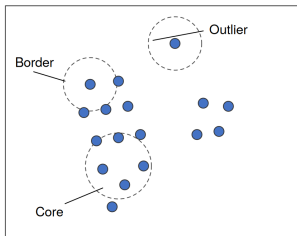




DBSCAN: Density-Based Spatial Clustering of Applications with Noise

- Bergantung pada gagasan kluster berbasis kepadatan: kluster didefinisikan sebagai set maksimal titik *density-connected*
- Menemukan kelompok bentuk tidak teratur dalam basis data spasial dengan *noise*

Titik *border* memiliki lebih sedikit dari *MinPts* dalam *Eps*, tetapi berada di sekitar titik *core*



Titik *noise* adalah setiap titik yang bukan merupakan titik *core* atau titik *border*

Eps = 1 cm

MinPts = 5



DBSCAN: The Algorithm

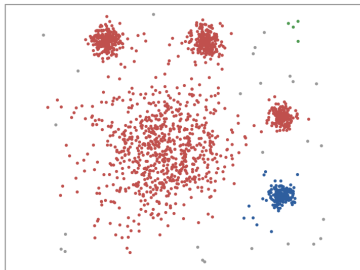
1. Secara acak memilih titik p
2. Ambil semua titik *density-reachable* dari p dengan mengacu Eps dan $MinPts$
3. Jika p adalah suatu titik *core*, sebuah kluster terbentuk
4. Jika p adalah titik *border*, tidak ada titik *density-reachable* dari p dan DBSCAN mengunjungi titik berikutnya dari basis data
5. Lanjutkan proses sampai semua titik telah diproses

If a spatial index is used, the computational complexity of DBSCAN is $O(n \log n)$, where n is the number of database objects. Otherwise, the complexity is $O(n^2)$.

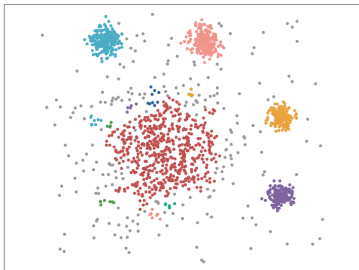


Kelemahan DBSCAN: Sensitive to Parameters

(a) MinPts = 4, Eps = 0.45 [3 kluster]

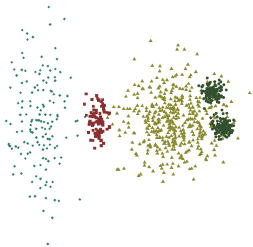


(b) MinPts = 4, Eps = 0.16 [16 kluster]



Perubahan kecil pada *Eps* mengubah jumlah kluster secara drastis: pada *Eps* besar seluruh titik melebur menjadi sedikit kluster, sedangkan pada *Eps* kecil kluster pecah menjadi banyak potongan dan makin banyak titik dianggap *noise*.

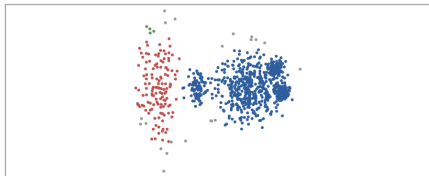
Kelemahan: DBSCAN tidak baik pada data berdimensi tinggi dan kepadatan yang bervariasi



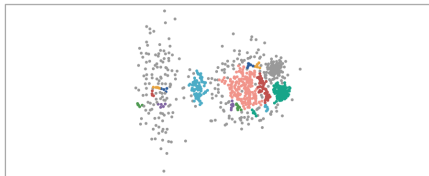
Original Points

- Varying densities
- High-dimensional data

(MinPts = 4, Eps = 0.40)

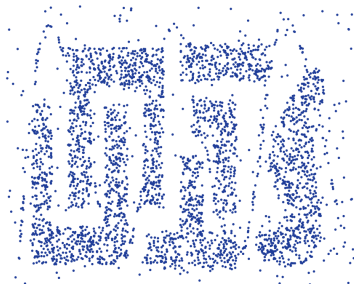


(MinPts = 4, Eps = 0.15)

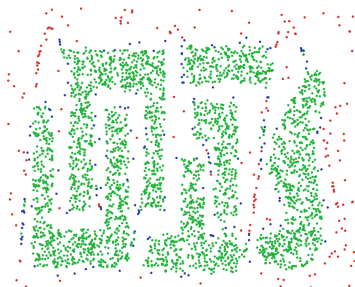




DBSCAN: Core, Border and Noise Points



Original Points

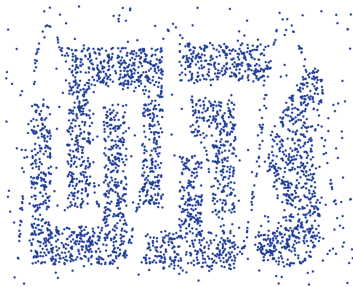


Point types: **core**, **border** and
noise

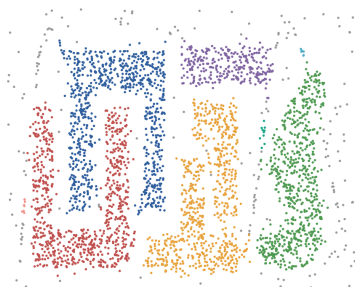
Eps = 12, MinPts = 8



DBSCAN bekerja dengan baik



Original Points



Clusters

- **Resistant to Noise**
- **Can handle clusters of different shapes and sizes**

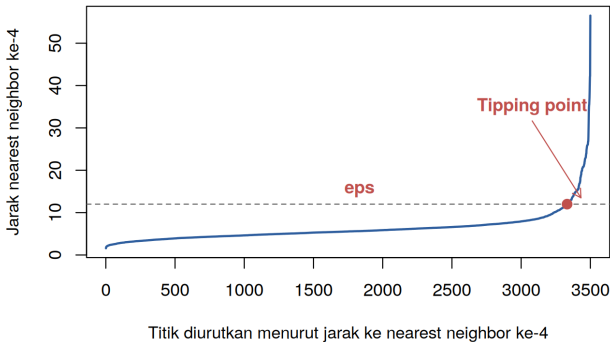


OPTICS: A Cluster-Ordering Method (1999)

- OPTICS: *Ordering Points To Identify the Clustering Structure*
 - Ankerst, Breunig, Kriegel, and Sander (SIGMOD'99)
 - Menghasilkan urutan khusus dari basis data dengan struktur pengelompokan berbasis kepadatannya
 - Urutan kluster ini berisi info ekuivalen ke pengelompokan berbasis kepadatan yang sesuai dengan berbagai pengaturan parameter
 - Baik untuk analisis kluster otomatis dan interaktif, termasuk menemukan struktur *clustering* intrinsik
 - Dapat direpresentasikan secara grafis atau menggunakan teknik visualisasi

DBSCAN: Determining EPS and MinPts

- Idanya adalah bahwa untuk titik-titik dalam sebuah kluster, *nearest neighbors* ke- k berada pada jarak yang kira-kira sama
- Titik *noise*: *nearest neighbor* ke- k pada jarak yang lebih jauh
- Jadi, plot diurutkan jarak setiap titik ke *nearest neighbor* ke- k





OPTICS: Some Extension from DBSCAN

- *Index-based*: $k = \#$ of dimensions, $N = \#$ of points
 - Complexity: $O(N \log N)$
- Jarak *core* dari suatu objek p : nilai terkecil ε sedemikian rupa sehingga ε -neighborhood p memiliki setidaknya $MinPts$ objek

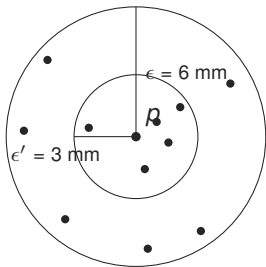
Misalkan $N_\varepsilon(p)$: ε -neighborhood dari p , ε adalah nilai jarak

$$\text{Core-distance}_{\varepsilon, MinPts}(p) = \begin{cases} \text{Tidak terdefinisi} & \text{jika } |N_\varepsilon(p)| < MinPts \\ \text{MinPts-distance}(p) & \text{lainnya} \end{cases}$$

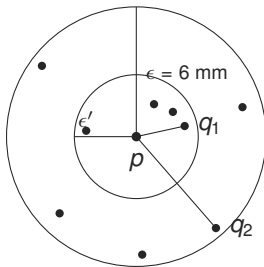
- *Reachability Distance* objek p dari *core object* q adalah nilai radius minimum yang membuat p *density-reachable* dari q

$$\text{Reachability-distance}_{\varepsilon, MinPts}(p, q) = \begin{cases} \text{Tidak terdefinisi} & \text{jika } q \text{ bukan objek core} \\ \max(\text{core-distance}(q), \text{distance}(q, p)) & \text{lainnya} \end{cases}$$

Core Distance & Reachability Distance



Core-distance of p



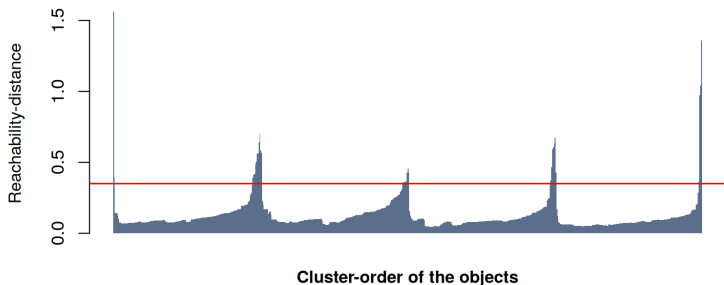
Reachability-distance $(p, q_1) = \epsilon' = 3 \text{ mm}$

Reachability-distance $(p, q_2) = d(p, q_2)$

Figure 10.16: OPTICS terminology. Based on [ABKS99].



OPTICS: Reachability Plot

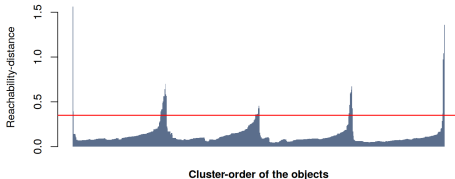
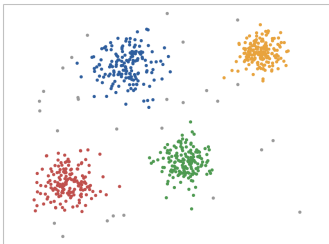


Setiap “lembah” pada plot *reachability* berkaitan dengan satu kluster. Memotong plot pada nilai ϵ' tertentu (garis merah) setara dengan menjalankan DBSCAN dengan $Eps = \epsilon'$.



Density-Based Clustering: OPTICS & Applications

<http://www.dbs.informatik.uni-muenchen.de/Forschung/KDD/Clustering/OPTICS/Demo>



Urutan kluster (*cluster-order*) hasil OPTICS

Struktur kluster pada ruang data

Kluster yang bersarang dan berbeda kepadatan tetap terbaca pada plot *reachability*, sesuatu yang tidak dapat ditangani DBSCAN dengan satu nilai Eps.



DBScan dan Optics di R

Menggunakan package **dbscan**

```
dbscan(x, eps, minPts = 5, weights = NULL,  
       borderPoints = TRUE, ...)  
  
optics(x, eps, minPts = 5, ...)
```

<https://cran.r-project.org/web/packages/dbscan/vignettes/dbscan.pdf>



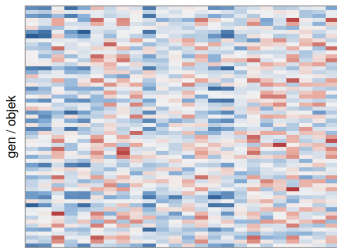
Bi-Clustering



Bi-Clustering: Mengapa Diperlukan?

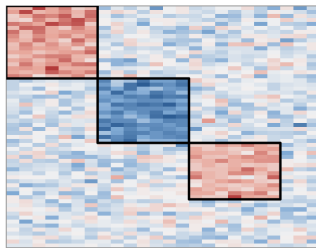
- Metode kluster sebelumnya mengelompokkan **baris saja** (objek) dengan memakai **seluruh kolom** (variabel)
- Pada data berdimensi tinggi, sebuah kelompok sering hanya konsisten pada **sebagian variabel** — variabel lain justru menjadi derau yang menutupi struktur
- **Bi-clustering** (Hartigan, 1972; Cheng & Church, 2000) mengelompokkan **baris dan kolom secara bersamaan**: yang dicari adalah *submatriks* yang koheren

Matriks data apa adanya



kondisi / variabel

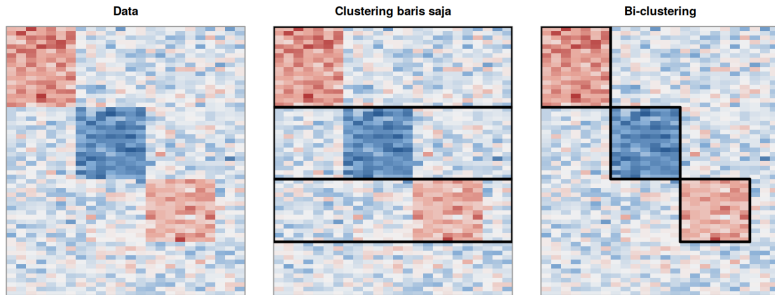
Setelah baris & kolom ditata ulang



kondisi / variabel

Blok yang sama sekali tidak terlihat pada matriks asli menjadi jelas setelah baris dan kolom ditata ulang.

Clustering Satu Arah vs Bi-Clustering



- **Clustering baris:** setiap objek masuk tepat satu klaster, dan klaster didefinisikan atas *semua* variabel
- **Bi-clustering:** satu objek boleh menjadi anggota beberapa bicluster (atau tidak sama sekali), dan tiap bicluster punya *himpunan variabelnya sendiri*
- Konsekuensinya: bi-clustering bersifat *lokal*, sedangkan clustering biasa bersifat *global*



Definisi dan Notasi

Misalkan A matriks data berukuran $n \times m$ dengan himpunan baris $X = \{x_1, \dots, x_n\}$ dan himpunan kolom $Y = \{y_1, \dots, y_m\}$.

Sebuah **bicluster** adalah pasangan (I, J) dengan $I \subseteq X$ dan $J \subseteq Y$, yang menghasilkan submatriks A_{IJ} sedemikian rupa sehingga A_{IJ} memenuhi kriteria *homogenitas* atau *koherensi* tertentu.

- $a_{iJ} = \frac{1}{|J|} \sum_{j \in J} a_{ij}$ rata-rata baris i pada kolom-kolom J
- $a_{Ij} = \frac{1}{|I|} \sum_{i \in I} a_{ij}$ rata-rata kolom j pada baris-baris I
- $a_{IJ} = \frac{1}{|I||J|} \sum_{i \in I} \sum_{j \in J} a_{ij}$ rata-rata seluruh submatriks

Masalah menemukan bicluster maksimal dengan kriteria tertentu terbukti *NP-hard*, sehingga hampir semua algoritma bersifat heuristik.



Tipe-Tipe Bi-Cluster

Nilai konstan

2	2	2	2
2	2	2	2
2	2	2	2
2	2	2	2

Baris konstan

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

Kolom konstan

1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

Koheren aditif

1	2	5	0
2	3	6	1
4	5	8	3
5	6	9	4

Koheren multiplikatif

1	2	5	0.5
2	4	10	1
4	8	20	2
5	10	25	2.5

Koheren evolusi (urutan)

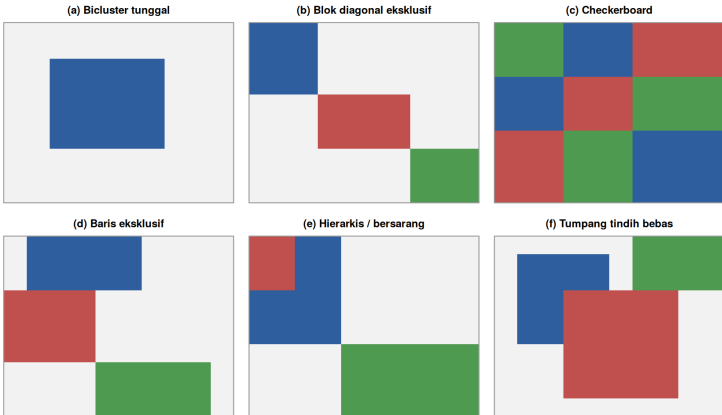
11	15	21	30
3	7	12	19
6	10	14	25
2	4	9	17

Aditif: $a_{ij} = \mu + \alpha_i + \beta_j$ Multiplikatif: $a_{ij} = \mu \cdot \alpha_i \cdot \beta_j$
antar kolom yang harus sama (*order-preserving*).

Evolusi: hanya *urutan* nilai
Madeira & Oliveira (2004)



Struktur Bi-Cluster dalam Matriks Data



Pilihan algoritma bergantung pada struktur yang diasumsikan: metode *spectral* mengasumsikan struktur *checkerboard*, sedangkan Cheng & Church, Plaid, dan Bimax memungkinkan bicluster yang tumpang tindih bebas.



Ukuran Kualitas: Mean Squared Residue

Cheng & Church (2000) mengukur koherensi submatriks A_{IJ} melalui **residu** setiap sel:

$$r_{ij} = a_{ij} - a_{iJ} - a_{IJ} + a_{IJ}$$

dan **Mean Squared Residue** (MSR) dari bicluster (I, J) :

$$H(I, J) = \frac{1}{|I||J|} \sum_{i \in I} \sum_{j \in J} r_{ij}^2$$

- $H(I, J) = 0 \Leftrightarrow$ submatriks *sempurna koheren aditif*: $a_{ij} = \mu + \alpha_i + \beta_j$
- Submatriks (I, J) disebut **δ -bicluster** jika $H(I, J) \leq \delta$ untuk suatu ambang $\delta \geq 0$
- Tujuan: menemukan δ -bicluster dengan *ukuran sebesar mungkin* (baris $\times$ kolom terbanyak)
- Kelemahan: MSR cenderung memilih bicluster dengan variansi rendah; sering ditambahkan syarat variansi baris minimum



Contoh Perhitungan MSR

Submatriks A_{IJ}				
	y_1	y_2	y_3	a_{iJ}
x_1	10	20	5	11,67
x_2	12	22	13	15,67
x_3	14	24	9	15,67
x_4	16	26	11	17,67
a_{iJ}	13	23	9,5	15,17

Tanpa perubahan ($a_{23} = 7$) submatriks ini sempurna aditif sehingga $H = 0$.

Matriks residu r_{ij}				
	y_1	y_2	y_3	$\overline{r^2}$
x_1	0,5	0,5	-1	0,50
x_2	-1,5	-1,5	3	4,50
x_3	0,5	0,5	-1	0,50
x_4	0,5	0,5	-1	0,50
$\overline{r^2}$	0,75	0,75	3,00	

$H(I, J) = 1,5$. Baris x_2 dan kolom y_3 menyumbang residu terbesar $\Rightarrow$ kandidat pertama untuk dihapus.

Contoh: $r_{23} = a_{23} - a_{2J} - a_{I3} + a_{IJ} = 13 - 15,67 - 9,5 + 15,17 = 3$, dan $H(I, J)$ adalah rata-rata r_{ij}^2 atas seluruh sel.



Algoritma Cheng & Church: Kerangka Umum

1. **Multiple node deletion** — untuk matriks besar, hapus *sekaligus* semua baris (kolom) yang residunya melebihi $\alpha \cdot H(I, J)$, dengan $\alpha > 1$
2. **Single node deletion** — lanjutkan dengan menghapus satu baris/kolom dengan residu terbesar sampai $H(I, J) \leq \delta$
3. **Node addition** — tambahkan kembali baris/kolom yang tidak menaikkan $H(I, J)$, agar bicluster sebesar mungkin
4. **Masking** — ganti sel bicluster yang sudah ditemukan dengan nilai acak, lalu ulangi dari awal untuk mencari bicluster berikutnya

Kompleksitas: $O(mn)$ per iterasi penghapusan; parameter yang harus dipilih adalah δ (ambang MSR), α (agresivitas penghapusan ganda), dan jumlah bicluster yang dicari.



Cheng & Church: Detail Penghapusan Node

Single node deletion

Hitung untuk setiap baris dan kolom:

$$d(i) = \frac{1}{|J|} \sum_{j \in J} r_{ij}^2, \quad e(j) = \frac{1}{|I|} \sum_{i \in I} r_{ij}^2$$

Hapus baris atau kolom dengan nilai terbesar, ulangi sampai $H(I, J) \leq \delta$.

Multiple node deletion ($\alpha > 1$)

Hapus *semua* baris dengan

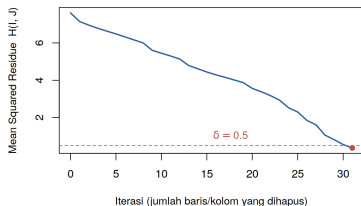
$d(i) > \alpha H(I, J)$ sekaligus, lalu semua kolom dengan $e(j) > \alpha H(I, J)$. Jauh lebih cepat untuk matriks besar.

```
msr <- function(A) {  
  aiJ <- rowMeans(A)  
  aIj <- colMeans(A)  
  aIJ <- mean(A)  
  R <- A - outer(aiJ, rep(1, ncol(A))) -  
    outer(rep(1, nrow(A)), aIj) + aIJ  
  mean(R^2)  
}  
  
# residu per baris dan per kolom  
d_baris <- rowMeans(R^2)  
e_kolom <- colMeans(R^2)
```

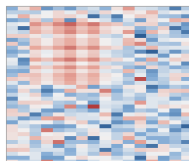
Node addition: baris $i \notin I$ ditambahkan bila $\frac{1}{|J|} \sum_{j \in J} (a_{ij} - a_{iJ} - a_{ij} + a_{iJ})^2 \leq H(I, J)$.



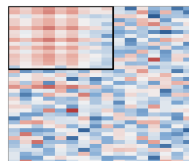
Ilustrasi δ -Biclustering



Matriks awal (bicluster tersembunyi)



Bicluster hasil δ -biclustering



Matriks 40×16 berisi satu bicluster aditif 16×8 yang tersembunyi di dalam derau. MSR turun tajam setiap kali baris/kolom paling tidak koheren dihapus, dan berhenti ketika $H(I, J) \leq \delta$. Algoritma menemukan kembali 15 dari 16 baris dan seluruh 8 kolom yang ditanam.



Algoritma Bi-Clustering Lainnya

Algoritma	Kriteria bicluster	Catatan
Cheng & Church (2000)	$MSR \leq \delta$ (koheren aditif)	Greedy, satu bicluster per putaran, perlu <i>masking</i>
Bimax (2006)	Blok konstan bernilai 1 pada matriks biner	Divide-and-conquer eksak; data harus dibinerkan
Plaid (2002)	Model aditif berlapis: $a_{ij} = \mu_0 + \sum_k (\mu_k + \alpha_{ik} + \beta_{jk}) \rho_{ik} \kappa_{jk}$	Mendukung bicluster tumpang tindih; berbasis model
Spectral (2003)	Struktur <i>checkerboard</i> via SVD	Cepat, tetapi mengasumsikan partisi baris dan kolom
xMotifs (2003)	Kolom dengan status/level yang sama	Cocok untuk data terdiskritisasi
QUBIC (2009)	Blok koheren pada graf berbobot	Skalabel untuk matriks besar

Semuanya tersedia pada package **biclust** di R kecuali QUBIC (package **QUBIC**).



Bi-Clustering dan Subspace Clustering

- Keduanya mencari kluster yang hanya berlaku pada *sebagian variabel* — keduanya menjawab masalah *curse of dimensionality*
- **Subspace clustering** (mis. CLIQUE): fokus pada objek yang *berdekatan* dalam suatu subruang; hasilnya berupa kluster objek beserta subruangnya
- **Bi-clustering**: fokus pada *pola koherensi nilai* pada submatriks; nilai boleh berjauhan asalkan berubah secara serentak (koheren aditif/multiplikatif)
- Bi-clustering lazim disebut juga *co-clustering*, *two-mode clustering*, atau *block clustering*

Gunakan bi-clustering ketika baris *dan* kolom sama-sama ingin dikelompokkan, ketika kluster diperkirakan tumpang tindih, atau ketika hanya sebagian variabel yang relevan untuk tiap kelompok.



Evaluasi Hasil Bi-Clustering

- **Validasi internal**
 - MSR $H(I, J)$ — makin kecil makin koheren
 - Variansi baris: $\text{VAR}(I, J) = \frac{1}{|I||J|} \sum_{i,j} (a_{ij} - a_{iJ})^2$, untuk menghindari bicluster konstan yang trivial
 - Ukuran bicluster (jumlah baris $\times$ kolom) dan tingkat tumpang tindih antar bicluster
- **Validasi eksternal** (bila kelompok sebenarnya diketahui)
 - *Recovery* dan *relevance* berbasis indeks Jaccard antara bicluster hasil dan bicluster acuan
 - Pada data ekspresi gen: pengayaan (*enrichment*) anotasi fungsional dalam bicluster
- **Stabilitas**: seberapa konsisten bicluster bertahan pada *resampling* atau perubahan parameter δ dan α



Bi-Clustering di R: package *biclust*

```
library(biclust)

# Cheng & Church
res <- biclust(G, method = BCCC(),
              delta = 0.2, alpha = 1.5,
              number = 3)

res
## Number of Clusters found: 3
## First 3 Cluster sizes:
##           BC 1 BC 2 BC 3
## Number of Rows:    14   16  13
## Number of Columns:    6    6   7

# keanggotaan
which(res@RowxNumber[, 1]) # baris bicluster 1
which(res@NumberxCol[1, ]) # kolom bicluster 1
```

```
# algoritma lain
biclust(binarize(G), method = BCBimax(),
        minr = 4, minc = 4, number = 10)
biclust(G, method = BCPlaid())
biclust(G, method = BCSpectral(),
        numberOfEigenvalues = 3)

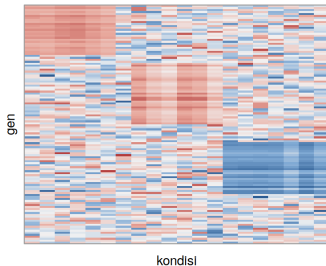
# visualisasi
drawHeatmap(G, res, number = 1)
parallelCoordinates(G, res, number = 1)
bubbleplot(G, res)
```

Standardisasi kolom sebelum bi-clustering, dan pilih δ relatif terhadap variansi data — δ terlalu besar menghasilkan bicluster besar yang tidak bermakna.

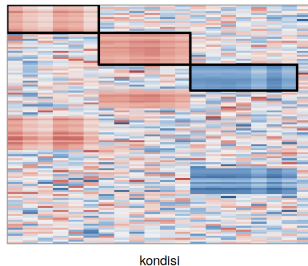


Studi Kasus 1: Data Ekspresi Gen

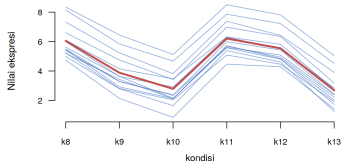
Data ekspresi gen (120 gen x 20 kondisi)



Hasil BCCC: 3 bicluster



Profil gen anggota bicluster 1



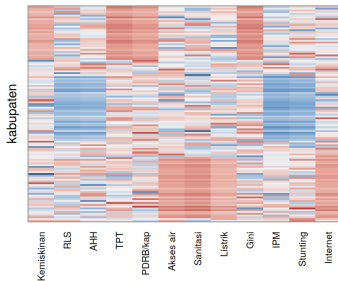
Matriks $120 \text{ gen} \times 20 \text{ kondisi}$ memuat tiga blok koheren. BCCC dengan $\delta = 0,2$ menemukan ketiganya dengan himpunan kondisi yang tepat.

Profil gen anggota bicluster 1 bergerak *sejajar* antar kondisi — inilah arti koheren aditif; garis merah adalah profil rata-rata bicluster.

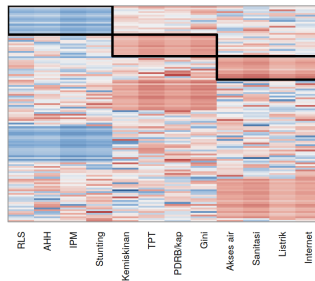


Studi Kasus 2: Kabupaten × Indikator Sosial-Ekonomi

120 kabupaten x 12 indikator (z-score)



Hasil BCCC: 3 bicluster



BCCC menemukan tiga bicluster yang masuk akal secara substantif: (1) kelompok kabupaten yang seragam pada *RLS*, *AHH*, *IPM*, *Stunting* (dimensi pendidikan–kesehatan); (2) kelompok pada *Kemiskinan*, *TPT*, *PDRB/kapita*, *Gini* (dimensi ekonomi); (3) kelompok pada *Akses air*, *Sanitasi*, *Listrik*, *Internet* (dimensi infrastruktur dasar). Satu kabupaten dapat masuk lebih dari satu bicluster — sesuatu yang tidak mungkin pada K-Means.



Ringkasan Bi-Clustering

- Bi-clustering mengelompokkan **baris dan kolom serentak**; hasilnya submatriks koheren, bukan partisi objek
- Cocok untuk matriks berdimensi tinggi di mana tiap kelompok hanya relevan pada sebagian variabel, dan kelompok boleh tumpang tindih
- Ukuran koherensi yang paling umum adalah **Mean Squared Residue**; δ -bicluster adalah submatriks dengan $H(I, J) \leq \delta$
- Algoritma Cheng & Church bersifat greedy: *node deletion* $\rightarrow$ *node addition* $\rightarrow$ *masking*
- Pilihan algoritma mengikuti struktur bicluster yang diasumsikan; di R tersedia lewat package **biclust**
- Hasil harus divalidasi (MSR, variansi, ukuran, stabilitas) dan **diinterpretasi secara substantif** — bicluster yang koheren secara statistik belum tentu bermakna



Evaluation of Clustering



Chapter 10. Cluster Analysis: Basic Concepts and Methods

- Cluster Analysis: Basic Concepts
- Partitioning Methods
- Hierarchical Methods
- Density-Based Methods
- Bi-Clustering
- **Evaluation of Clustering** 
- Summary



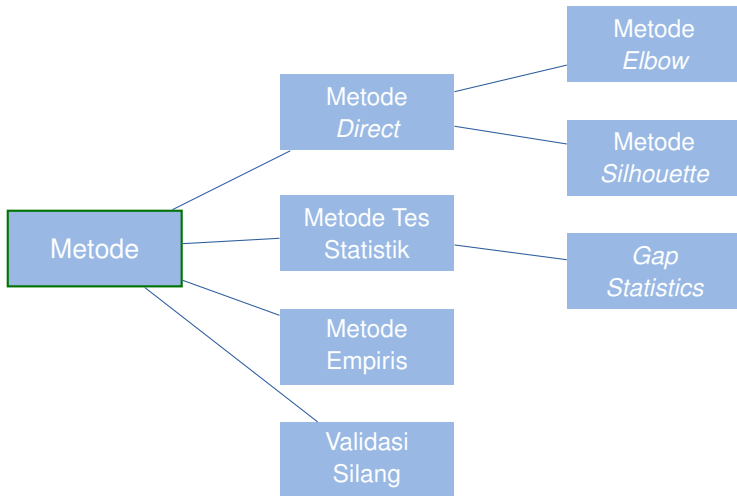
Estimasi Jumlah Cluster Optimal

Untuk mencari jumlah kluster yang optimal pada *partitional clustering* (K-Means, K-Medoids, CLARA), direkomendasikan untuk memilihnya berdasarkan:

1. Konteks masalah yang dihadapi, misalnya jika Anda tahu bahwa ada sejumlah grup tertentu dalam data Anda (Anda memiliki ekspektasi atau hipotesis yang kuat, namun opsi ini subjektif)
 2. Metode *direct* atau statistik
-



Estimasi Jumlah Cluster Optimal





Estimasi Jumlah Klaster

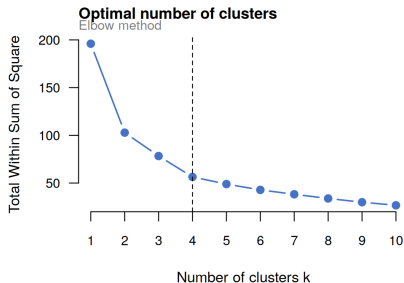
- Metode empiris
 - Jumlah klaster $\approx \sqrt{n/2}$ untuk *dataset* dengan n objek
- Metode *Direct* dengan *Elbow*
 - Gunakan titik balik dalam kurva jumlah dalam varians klaster berhubungan dengan jumlah klaster
- Metode validasi silang
 - Bagilah kumpulan data tertentu menjadi m bagian
 - Gunakan $m - 1$ bagian untuk mendapatkan model pengelompokan
 - Gunakan bagian yang tersisa untuk menguji kualitas pengelompokan
 - Misalnya, untuk setiap titik dalam set tes, temukan sentroid terdekat, dan gunakan jumlah jarak kuadrat antara semua titik dalam set tes dan sentroid terdekat untuk mengukur seberapa baik model sesuai dengan set tes
 - Untuk setiap $k > 0$, ulangi m kali, bandingkan ukuran kualitas keseluruhan dengan k yang berbeda, dan temukan jumlah klaster yang paling sesuai dengan data



Estimasi Jumlah Cluster Optimal

1. Elbow method

Metode ini melihat nilai total jumlah kuadrat dalam kluster (*within-cluster sum of square/WSS*) sebagai fungsi dari jumlah kluster.



```
# load required packages
library(factoextra)
library(NbClust)

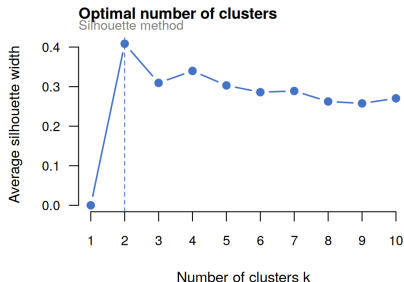
# Elbow method
fviz_nbclust(Eurojobs, kmeans,
             method = "wss") +
  geom_vline(xintercept = 4,
            linetype = 2) +
  labs(subtitle =
        "Elbow method")
```

Lokasi *elbow* di plot biasanya dianggap sebagai indikator jumlah kluster yang sesuai karena itu berarti menambahkan kluster lain tidak meningkatkan partisi yang jauh lebih baik. Metode *Elbow* terkadang ambigu dan alternatifnya adalah *Average silhouette method*.



Estimasi Jumlah Cluster Optimal

2. Average silhouette method



```
# load required packages
library(factoextra)
library(NbClust)

# Silhouette method
fviz_nbclust(Eurojobs, kmeans,
  method = "silhouette") +
  labs(subtitle =
    "Silhouette method")
```

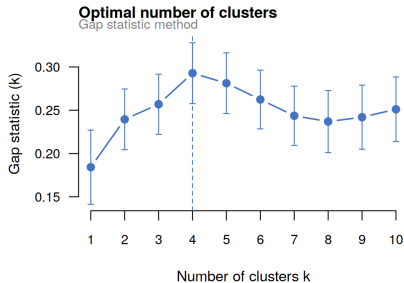
Jumlah optimal cluster k adalah nilai k yang memaksimalkan nilai *average silhouette* pada kisaran nilai yang mungkin untuk k (Kaufman dan Rousseeuw [1990]).

Metode *Silhouette* mengukur kualitas pengelompokan dan menentukan seberapa baik setiap titik terletak di dalam klasternya. Metode ini menghitung *average silhouette* pengamatan untuk berbagai nilai k .



Estimasi Jumlah Cluster Optimal

3. Gap statistic method



Nilai k yang memaksimalkan *gap statistic* (atau yang menghasilkan *gap statistics* yang terbesar) merupakan estimasi jumlah kluster optimal.

```
# load required packages
library(factoextra)
library(NbClust)

# Gap statistic
set.seed(42)
fviz_nbclust(Eurojobs, kmeans,
              nstart = 25,
              method = "gap_stat",
              nboot = 500) +
  labs(subtitle =
        "Gap statistic method")
```

Pendekatan ini dapat diterapkan pada metode *clustering* apa pun. Statistik ini membandingkan total dalam variasi *intra-cluster* untuk nilai k yang berbeda dengan nilai ekspektasi di bawah distribusi referensi nol dari data.



EVALUASI MODEL

Ketika saya mencoba sebuah metode *clustering* pada sebuah *dataset*, bagaimana caranya mengevaluasi apakah hasil pengelompokan itu bagus?



EVALUASI MODEL

Evaluasi Model Analisis Kluster

1. Evaluasi tendensi pengelompokan

Jika suatu *dataset* terdiri dari individu-individu yang seragam/homogen/berasal dari distribusi *uniform*, apakah pengelompokan masih dibutuhkan?

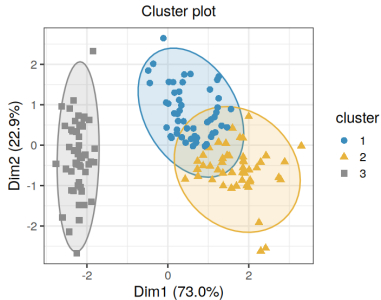
2. Mengukur kualitas pengelompokan

Seberapa bagus hasil pengelompokan yang dilakukan? Bisa menggunakan validasi internal, eksternal dan relatif.

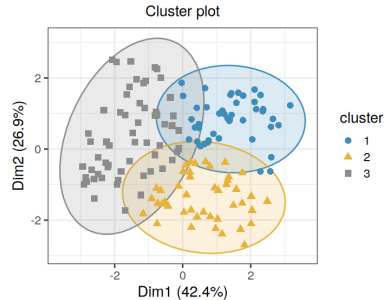


Evaluasi Tendensi Pengelompokan

Pendahuluan



Pada data “**iris**” ini terlihat bahwa pengelompokan bisa dilakukan dan analisis kluster menghasilkan kelompok yang **bermakna**



Pada data “**random_df**” yang menyebar merata (*uniform*) ini, analisis kluster bisa dipaksakan namun kelompok yang terbentuk **tidak bermakna**

Note: Evaluasi tendensi pengelompokan harus dilakukan untuk memeriksa validitas dari analisis kluster, “apakah *dataset* dapat dikelompokkan secara bermakna?”



Evaluasi Tendensi Pengelompokan

Hopkins Statistics

- Analisis kluster mensyaratkan data berdistribusi *non-uniform* (Han, Kamber, and Pei, 2012)
- *Hopkins Statistics* digunakan untuk memeriksa tendensi pengelompokan suatu *dataset* dengan mengukur peluang suatu data berasal dari distribusi seragam (*uniform*)
- H_0 : *dataset* memiliki distribusi seragam vs H_1 : *dataset* tidak memiliki distribusi seragam
- Statistik Uji:
$$H = \frac{\sum_{i=1}^n y_i}{\sum_{i=1}^n x_i + \sum_{i=1}^n y_i}$$
- Jika $H > 0,75$ mengindikasikan bahwa terdapat tendensi pengelompokan pada CI 90%
- Aplikasi pada R: menggunakan `get_clust_tendency()` pada package `factoextra`

```
library(factoextra)
res <- get_clust_tendency(df,
  n = nrow(df)-1, graph = FALSE)
res$hopkins_stat
## [1] 0.818

res <- get_clust_tendency(random_df,
  n = nrow(random_df)-1, graph = FALSE)
res$hopkins_stat
## [1] 0.466
```

- Data pertama cocok untuk dilakukan analisis kluster karena nilai $H = 0,818$ (lebih besar dari *threshold*)
- Data kedua tidak cocok untuk dikelompokkan karena nilai $H = 0,466$ (lebih kecil dari *threshold*)

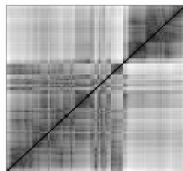


Evaluasi Tendensi Pengelompokan

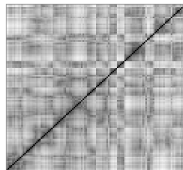
Visual Assessment of cluster Tendency (VAT Algorithm)

Algoritma VAT (Bezdek and Hathaway, 2002):

1. Hitung matriks *dissimilarity* (DM) antar objek *dataset* dengan menghitung jarak Euclidian
2. Urutkan DM agar objek yang mirip saling berdekatan satu sama lain. Tahapan ini menghasilkan matriks *dissimilarity* yang sudah tersusun – *Ordered Dissimilarity Matrix* (ODM)
3. Output algoritma VAT adalah visualisasi dari ODM yang digambarkan dalam sebuah *Ordered Dissimilarity Image* (ODI)



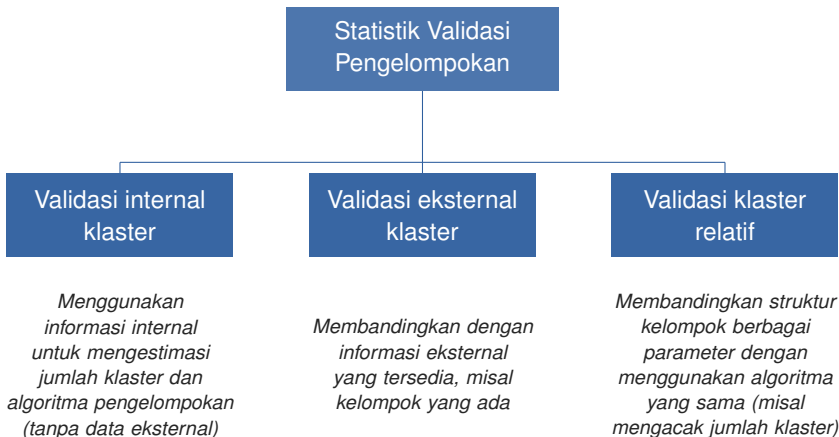
ODI data “iris”: blok gelap = kluster



ODI data “random_df”: tanpa blok



Mengukur Kualitas Pengelompokan





Mengukur Kualitas Pengelompokan

Validasi Internal Klaster

Ingat Tujuan Analisis Klaster!

1. *Objek di klaster yang sama harus semirip mungkin*
2. *Objek di klaster yang berbeda harus mudah dibedakan*

Validasi Internal Klaster mengukur:

1. ***Compactness***: *seberapa dekat objek-objek di dalam klaster yang sama (lower within cluster variation)*
2. ***Separation***: *seberapa baik pembeda sebuah klaster dengan klaster lainnya*
3. ***Connectivity***: *sejauh apa objek-objek yang tergolong dalam satu klaster yang sama berperan sebagai tetangga terdekat mereka (semakin minimum semakin baik)*



Mengukur Kualitas Pengelompokan

Validasi Internal Klaster: *Silhouette Index* & *Dunn Index*

Silhouette Index

- Mengukur seberapa dekat tiap-tiap objek di satu klaster dengan objek di klaster tetangga
- Nilai S mendekati 1 menunjukkan bahwa objek tersebut *well-clustered*
- Nilai S mendekati -1 menunjukkan bahwa objek tersebut *poorly-clustered*

R script:

```
km.res <- eclust(df, "kmeans",  
  k = 3, nstart = 25,  
  graph = FALSE)  
silinfo <- km.res$silinfo
```

Dunn Index

- Mengukur perbandingan minimum *intercluster separation* dengan maximum *intra-cluster distance*
- Nilai D yang tinggi menunjukkan pengelompokan yang bagus

R script:

```
library(fpc)  
km_stats <- cluster.stats(  
  res.dist, km.res$cluster)  
km_stats$dunn
```



Mengukur Kualitas Pengelompokan

Validasi Eksternal Klaster

- Pada metode ini, hasil pengelompokan dibandingkan dengan klasifikasi yang sudah tersedia.
- Contoh: membandingkan hasil pengelompokan tanaman pada data "Iris" dengan variabel "Species"

```
table(iris$Species,  
      km.res$cluster)
```

```
##           1  2  3  
## setosa    50  0  0  
## versicolor 0 11 39  
## virginica  0 36 14
```

Corrected Rand Index dan *VI* mengukur kemiripan antara dua *partitions*. Rentang nilainya antara -1 (*no agreement*) dan 1 (*perfect agreement*).

```
library("fpc")  
# Compute cluster stats  
species <- as.numeric(iris$Species)  
clust_stats <- cluster.stats(  
  d = dist(df), species,  
  km.res$cluster)
```

```
# Corrected Rand index  
clust_stats$corrected.rand  
## [1] 0.62
```

```
# VI  
clust_stats$vi  
## [1] 0.748
```



Summary

Tahapan Analisis Kluster

